Manual técnico: Oruro Trabaja

Bolsa de trabajo digital de la Gobernación Autónoma Departamental de Oruro (GADOR)
Versión 1.0 · Noviembre 2026

---

1. Propósito

Oruro Trabaja es la bolsa de trabajo digital del departamento de Oruro. El sistema permite a los postulantes registrar su perfil y hoja de vida, a empresas publicar ofertas laborales y a GADOR administrar y auditar todo el flujo mediante un panel de gestión.
Este manual describe la implementación técnica para el equipo de evaluación y desarrollo: stack, arquitectura, modelo de datos, seguridad y operación.

2. Stack

Componente	Tecnología
Backend	Django 6.1 (Python 3.12)
Base de datos	PostgreSQL 15 (producción) · SQLite (desarrollo)
Frontend	Django Templates + Tailwind CSS 4 + HTMX
Iconografía	Material Symbols (auto-hospedado, woff2)
Archivos	Cloudflare R2 (compatible S3) o disco local
SMS	Textbee (API) con modo demostración integrado
Autenticación	Sesiones Django, contraseñas con bcrypt
Servidor WSGI	Gunicorn (1 worker + threads)
Estáticos en producción	WhiteNoise (compresión + manifest con hash)
PWA	Manifest + Service Worker + página offline

3. Arquitectura

Monolito modular con dependencias unidireccionales: los módulos de **plataforma** (cuentas, auditoría) y el kernel compartido no dependen del dominio de negocio; el dominio no depende de la capa de presentación. Los detalles y reglas de dependencia se detallan en docs/ARQUITECTURA.md.

```
oruro_trabaja/
├── config/                    # settings (base/dev/prod), urls, wsgi, asgi
├── core/                      # kernel transversal
│   ├── models.py              #   modelos base (TimeStampedModel)
│   ├── permissions.py         #   decorador role_required
│   ├── pagination.py          #   paginado estándar
│   ├── normalizers.py         #   normalización de celulares (E.164)
│   ├── storage.py             #   storage privado configurado por entorno
│   ├── context_processors.py
│   └── exceptions.py          #   DomainError / Conflict
├── apps/
│   ├── platform/
│   │   ├── accounts/          # User, roles, OTP, registro, login, throttle
│   │   ├── audit/             # AuditLog (RF16) y panel de auditoría
│   │   └── notifications/     # servicio de notificaciones (extensible)
│   ├── recruitment/
│   │   ├── catalogos/         # profesiones, habilidades, idiomas, rubros…
│   │   ├── empresas/          # registro y validación de empresas
│   │   ├── postulantes/       # perfil, CV y completitud
│   │   └── ofertas/           # ofertas, postulaciones, guardadas
│   └── dashboard/             # dashboards por rol y reportes
├── templates/                 # plantillas por módulo + partials
├── static/                    # css compilado, js, fuentes, imágenes
└── assets/                    # fuente CSS (Tailwind) y material de seed
```

Patrones aplicados: **use cases** por operación de negocio (Registrar Empresa, Cambiar Estado Empresa, aprobar oferta, …), **servicios** (calcular completitud, audit.registrar), forms de Django con validación de dominio y un solo punto de URLs por app.

4. Modelo de datos

El esquema completo, sus relaciones y datos demo se entregan en:

- docs/oruro_trabaja.sql — dump de PostgreSQL con esquema y datos de demostración.
- docs/diagrama_er.png — diagrama entidad–relación.
- docs/diccionario_datos.md — diccionario de datos campo por campo.

Resumen por módulo:

Módulo	Tablas principales
Cuentas	users, roles, registros_pendientes, codigos_otp
Auditoría	audit_auditlog
Catálogos	profesiones, habilidades, idiomas, rubros, tipos_contrato, modalidades, departamentos
Empresas	empresas

Módulo	Tablas principales
Postulantes	postulante_profiles, formaciones, experiencias, postulante_habilidades, postulante_idiomas
Ofertas	ofertas, oferta_habilidades, postulaciones, ofertas_guardadas

5. Instalación y ejecución (desarrollo)

```
cp .env.example .env          # completar credenciales de seed
uv sync                       # entorno virtual + dependencias
npm install && npm run css:build   # solo si se modifican estilos

python manage.py migrate
python manage.py seed_catalogos
python manage.py seed_usuarios
python manage.py seed_empresas
python manage.py seed_ofertas

python manage.py runserver
```

Con estos comandos el sistema corre en SQLite con DEMO_MODE=True: el código OTP se muestra en la pantalla de verificación y no se envía SMS.

PostgreSQL

Definiendo POSTGRES_DB (más POSTGRES_USER/PASSWORD/HOST/PORT) en el entorno, Django usa PostgreSQL automáticamente; si la variable no existe, usa SQLite.
Restaurar el dump:

```
psql -U <usuario> -d <base> < docs/oruro_trabaja.sql
```

6. Configuración (.env)

Grupo	Variables	Uso
Base	SECRET_KEY, DEBUG, ALLOWED_HOSTS	núcleo
Demo	DEMO_MODE	True: OTP en pantalla, sin SMS
OTP	OTP_CODE_TTL_MINUTES, OTP_RESEND_SECONDS, OTP_MAX_ATTEMPTS, OTP_MAX_SENDS_PER_HOUR, OTP_PHONE_*, OTP_IP_*	límites de verificación
Login	LOGIN_MAX_ATTEMPTS=5, LOGIN_TIMEOUT_SECONDS=900	bloqueo por IP

Grupo	Variables	Uso
Registro	PENDING_REGISTRATION_TTL_MINUTES, EMPRESA_MAX_OFERTAS_ACTIVAS	reglas de negocio
Base de datos	POSTGRES_DB/USER/PASSWORD/HOST/PORT/SSLMODE, DB_CONN_MAX_AGE	PostgreSQL opcional
Storage	STORAGE_BACKEND=local|s3, AWS_*	archivos (Cloudflare R2)
SMS	TEXTBEE_API_KEY, TEXTBEE_DEVICE_ID, TEXTBEE_BASE_URL	envío real

Las credenciales nunca se versionan: .env está en .gitignore.

7. Seguridad

- **Contraseñas**: hash bcrypt (hashers de Django).
- **Bloqueo por IP en el login**: 5 intentos fallidos desde una IP → 15 minutos de espera; un login exitoso limpia el contador. No bloquea la cuenta.
- **OTP por celular**: código hash-eado, TTL de 10 minutos, máx. 5 intentos, reenvío cada 60 s y cupos por teléfono y por IP por ventana de tiempo.
- **RBAC por rol**: middleware de acceso (role_required) con roles POSTULANTE, EMPRESA, SUPERADMIN en la tabla roles.
- **CSRF** en todos los formularios; escape automático de plantillas (XSS).
- **Archivos**: los CV se guardan en storage privado y se descargan solo vía vista con permiso (URL firmada con expiración de 1 h en S3); los nombres de archivo colisionantes se resolven automáticamente (file_overwrite=False).
- **Producción**: HSTS, cookies Secure, X-Frame-Options: DENY, SECURE_SSL_REDIRECT, DEBUG=False (ver config/settings/prod.py).

8. Auditoría

Módulo apps.platform.audit:

- Tabla audit_auditlog: usuario, accion (CREAR, ACTUALIZAR, ELIMINAR, LOGIN, CAMBIO_ESTADO), tabla, registro_id, datos_antes/datos_despues (JSON), ip, fecha indexada.
- Helper audit.registrar(...) — una línea por punto instrumentado.
- Puntos activos: registro de empresa (CREAR), validación de empresa por GADOR (CAMBIO_ESTADO) y revisión de oferta (CAMBIO_ESTADO publicar/devolver).
- Panel **GADOR** → **Auditoría**: lista paginada y detalle con el JSON antes/después.

9. Pruebas

Suite de 159 tests Django (modelos, use cases, vistas, permisos, auditoría, storage, SMS):

```
python manage.py test --settings=config.settings.test
python manage.py check
python manage.py makemigrations --check --dry-run
```

Verificados en SQLite y en PostgreSQL 15 (Docker).

10. PWA

Manifest generado dinámicamente (/manifest.webmanifest), service worker (/sw.js) con precache del shell, y página offline (/offline/). Requiere HTTPS o localhost para registrarse.

11. Despliegue (resumen)

- gunicorn config.wsgi con **1 worker** (el throttle por IP usa la cache en proceso; escalar a Redis si se agregan workers).
- Estáticos: collectstatic + WhiteNoise (manifest con hash y compresión).
- Nginx como proxy inverso + TLS (certbot).
- .env de producción: DEBUG=False, SECRET_KEY nueva, ALLOWED_HOSTS, STORAGE_BACKEND=s3, bloque POSTGRES_*.

12. Limitaciones de la versión demo

- Cambio de contraseña y recuperación por correo: presentados en el panel "Mi cuenta" con estado **No disponible en demo** (marcados en el roadmap).
- Correos transaccionales (Brevo) y notificaciones push: no implementados.
- El throttle por IP usa cache en proceso: válido con un solo worker.